

ISYE 671: Linear Optimization and Network Flows — Lecture Notes

Solving Integer Programming Models

Instructor: Dr. Ziteng Wang | **Date:** February 16, 2026 | **Reading:** Rardin, Chapter 12

1. Why Is Integer Programming Hard?

Last week we formulated integer programming (IP) models — binary variables for yes/no decisions, switching constraints linking continuous and binary variables, big-M disjunctions, and so on. Today we ask: *how do we solve these models, and why is it so much harder than LP?*

1.1 The Fundamental Difficulty

Recall that the simplex method solves LPs efficiently by walking along edges of a convex polyhedron. The feasible region of an LP is a *convex set*, and any local optimum is a global optimum. Adding integrality constraints destroys this structure. The feasible region of an IP is a discrete set of points — there are no edges to walk along, no gradient to follow, and no guarantee that a “nearby” feasible solution is anywhere close to optimal.

Consider a binary program with n variables. The brute-force approach — enumerate all 2^n combinations — is hopelessly slow. With $n = 50$, we face over 10^{15} candidates. With $n = 100$, the number exceeds the number of atoms in the observable universe. Yet modern solvers routinely handle problems with thousands of binary variables. How?

The answer lies in a suite of techniques that exploit the *LP relaxation* as a guide. The core ideas are relaxation, bounding, branching, and cutting — all working together to prune the search space so that only a tiny fraction of the 2^n combinations are ever examined.

1.2 Computational Complexity

Integer programming is NP-hard in general. This means:

- No algorithm is known that solves all IP instances in polynomial time.
- The worst-case running time grows exponentially with problem size.
- Even determining whether a feasible integer solution *exists* is NP-complete.

By contrast, linear programming is solvable in polynomial time (via interior point methods) and in practice is extremely fast via the simplex method. The difference in computational difficulty between LP and IP is not incremental — it is a fundamental divide in the theory of computation.

However, NP-hardness is a *worst-case* statement. Many practical instances are solved efficiently thanks to problem structure that solvers can exploit. Understanding the solution methods helps us formulate models that are easier for solvers to handle.

1.3 LP vs. IP: A Visual Comparison

Consider a two-variable integer program:

$$\begin{aligned} \max \quad & 3x_1 + 2x_2 \\ \text{s.t.} \quad & 2x_1 + x_2 \leq 6 \end{aligned}$$

$$x_1 + 2x_2 \leq 6$$

$$x_1, x_2 \geq 0, \quad x_1, x_2 \in \mathbb{Z}$$

The LP relaxation feasible region is a polygon; the IP feasible region is the set of integer-coordinate points inside that polygon. The LP optimal solution $(2, 2)$ happens to be integer here — but that is a lucky coincidence, not the norm. When the LP optimal is fractional, we must work harder.

2. LP Relaxation and Bounds

2.1 What Is LP Relaxation?

Given an IP model, its **LP relaxation** is obtained by dropping all integrality constraints. Binary variables $y_j \in \{0, 1\}$ become $0 \leq y_j \leq 1$; general integer variables $x_j \in \mathbb{Z}_+$ become $x_j \geq 0$.

The LP relaxation is a *larger* problem — it has more feasible solutions (every integer solution is also LP-feasible, but not vice versa). Therefore:

- For a **minimization** IP: $z_{LP}^* \leq z_{IP}^*$ (the LP relaxation gives a **lower bound**).
- For a **maximization** IP: $z_{LP}^* \geq z_{IP}^*$ (the LP relaxation gives an **upper bound**).

In either case, the LP relaxation provides a **bound** on the best possible IP solution.

2.2 The Integrality Gap

The **integrality gap** measures how far apart the LP relaxation bound and the true IP optimum are:

$$\text{Gap} = \frac{|z_{IP}^* - z_{LP}^*|}{|z_{IP}^*|} \times 100\%$$

A small gap means the LP relaxation is a tight approximation of the IP — the solver has less searching to do. A large gap means the LP relaxation is loose, and the solver must explore more of the search tree.

Practical insight: The tightness of the LP relaxation depends on the formulation. Two mathematically equivalent IP formulations can have very different LP relaxations, leading to dramatically different solve times. This is why modeling choices matter even when the problem is “the same.”

2.3 Examples from Our Models

Recall from last week’s lecture:

- **Swedish Steel (LP):** 9,953.7 kroner → **Swedish Steel (IP with fixed charges):** 11,017.1 kroner. The gap reflects the cost of setup decisions that the LP relaxation can “partially” activate.
- **NASA Capital Budgeting:** The LP relaxation can “fractionally” select missions, spending partial budgets. The IP must commit fully to each mission.

We will compute these gaps explicitly in the notebook.

2.4 When Does LP Relaxation Solve the IP Exactly?

There are important special cases where the LP relaxation naturally produces integer solutions:

- **Totally unimodular (TU) constraint matrices:** If the constraint matrix A is TU and the right-hand side b is integer, then every vertex of the LP polyhedron is integer. Network flow problems (transportation, assignment, shortest path) have TU matrices — this is why our network LP models from earlier in the course produced integer solutions “for free.”
 - **Integrality of polyhedra:** The theory of total unimodularity tells us precisely when LP relaxation suffices. This is one reason network optimization is computationally easier than general IP.
-

3. Branch and Bound

3.1 The Core Idea

Branch and bound (B&B) is the workhorse algorithm for integer programming. It systematically divides the feasible region into subproblems (branching) while using LP relaxation bounds to prune subproblems that cannot contain the optimal solution (bounding).

The algorithm maintains two key quantities:

- **Best known feasible solution** (the “incumbent”) with objective value $\bar{z}$.
- **LP relaxation bounds** for each active subproblem.

3.2 Algorithm Outline

Step 1: Initialize. Solve the LP relaxation of the original IP. If the solution is integer, we are done. Otherwise, set the LP bound and begin branching.

Step 2: Branch. Select a variable x_j whose LP relaxation value $x_j^* = f$ is fractional. Create two subproblems:

- **Left child:** Add constraint $x_j \leq \lfloor f \rfloor$
- **Right child:** Add constraint $x_j \geq \lceil f \rceil$

For binary variables, this becomes $x_j = 0$ (left) and $x_j = 1$ (right).

Step 3: Bound. Solve the LP relaxation of each subproblem. Three outcomes:

- *Infeasible:* Prune this subproblem (no integer solution possible here).
- *LP bound worse than incumbent:* Prune (no improvement possible here).
- *LP solution is integer:* Update incumbent if this solution is better.
- *LP solution is fractional with promising bound:* Keep this subproblem for further branching.

Step 4: Repeat. Select the next active subproblem and branch. Continue until no active subproblems remain.

3.3 A Worked Example

Consider the following small IP:

$$\max 4x_1 + 5x_2$$

$$\text{s.t. } x_1 + x_2 \leq 5$$

$$6x_1 + 10x_2 \leq 45$$

$$x_1, x_2 \geq 0, \quad x_1, x_2 \in \mathbb{Z}$$

Node 0 (Root): LP relaxation gives $x_1 = 1.25$, $x_2 = 3.75$, $z_{LP} = 23.75$. Both variables are fractional.

Branch on x_2 (largest fractional part):

- **Node 1:** Add $x_2 \leq 3$. LP gives $x_1 = 2$, $x_2 = 3$, $z_{LP} = 23$. **Integer!** Set incumbent $\bar{z} = 23$.
- **Node 2:** Add $x_2 \geq 4$. LP gives $x_1 = 0.833$, $x_2 = 4$, $z_{LP} = 23.33$. Fractional, and bound $23.33 > 23$ (incumbent), so this node is worth exploring.

Node 1 is pruned by integrality (it produced an integer solution). We continue with Node 2.

Branch on x_1 at Node 2:

- **Node 3:** $x_1 \leq 0$, $x_2 \geq 4$. LP gives $x_1 = 0$, $x_2 = 4.5$, $z_{LP} = 22.5$. Bound $22.5 < 23$ (incumbent). **Pruned by bound** — no integer solution in this subregion can beat 23.
- **Node 4:** $x_1 \geq 1$, $x_2 \geq 4$. The constraints require $6(1) + 10(4) = 46 > 45$, so this subproblem is **infeasible**. **Pruned by infeasibility**.

No active nodes remain. **Optimal:** $x_1 = 2$, $x_2 = 3$, $z^* = 23$.

Observations: This small example illustrates all three pruning mechanisms: integrality (Node 1), bounding (Node 3), and infeasibility (Node 4). The B&B tree had only 5 nodes instead of the 36 integer points we would need to enumerate.

3.4 Branching Strategies

The order in which we select variables to branch on and subproblems to explore significantly affects performance:

- **Variable selection:** Branch on the variable with the largest fractional part, or use *strong branching* (trial-solve both children to pick the most informative split).
- **Node selection:** *Best-first* (pick the node with the best LP bound — explores the most promising areas first); *depth-first* (dive deep quickly to find feasible solutions early, reducing the incumbent).
- Modern solvers use sophisticated hybrid strategies that adapt during the search.

3.5 Pruning Rules

A subproblem is pruned (not explored further) when:

1. **Infeasibility:** The LP relaxation of the subproblem is infeasible.
2. **Bound:** The LP relaxation bound is no better than the current incumbent.
3. **Integrality:** The LP relaxation produces an integer solution (update incumbent if better).

The tighter the LP relaxation, the more effective pruning is — this is why formulation quality matters so much.

4. Cutting Planes and Branch-and-Cut

4.1 The Idea

Branch and bound can be slow when the LP relaxation is loose. **Cutting planes** are additional linear inequalities that tighten the LP relaxation without removing any integer-feasible solutions.

A valid cut is satisfied by all integer solutions but violated by some fractional LP solutions (ideally including the current LP optimal). Adding cuts shrinks the LP polyhedron toward the **convex hull** of integer solutions.

Several families of cuts are used in practice: **Gomory cuts** (derived mechanically from the LP tableau), **cover inequalities** for knapsack-type constraints, **flow cover cuts** for fixed-charge networks, and **clique inequalities** for conflict constraints. Modern solvers apply these automatically.

4.2 Branch and Cut

Modern MIP solvers combine cutting planes with branch and bound in an algorithm called **branch and cut**. At each B&B node, the solver generates and adds violated cuts to tighten the LP relaxation before deciding whether to branch. When you see solver output like “Cuts: 142 applied,” this is branch and cut at work.

5. Interpreting MIP Solver Output

5.1 What to Look For

When solving a MIP, the solver produces a log that tells you a great deal about the solution process. Here are the key elements:

LP relaxation bound: The objective value of the initial LP relaxation. This is the theoretical best the IP could achieve.

Incumbent / best feasible: The best integer-feasible solution found so far.

MIP gap: $\frac{|\text{best bound} - \text{incumbent}|}{|\text{incumbent}|} \times 100\%$. When this reaches zero (or your tolerance), the solver has proven optimality.

Nodes explored: How many B&B nodes were processed. More nodes = harder problem.

Cuts applied: Number and type of cutting planes added.

Time: Total solve time.

5.2 Reading a Gurobi Log

A typical Gurobi MIP log looks like:

Optimize a model with 9 rows, 29 columns and 59 nonzeros

Model fingerprint: 0x3a7b8c1d

Variable types: 14 continuous, 15 integer (15 binary)

...

Root relaxation: objective 8.234000e+03, 12 iterations, 0.00 seconds

	Nodes		Current Node			Objective Bounds			Work	
	Expl	Unexpl	Obj	Depth	IntInf	Incumbent	BestBd	Gap	It/Node	Time
	0	0	8234.0000	0	4	-	8234.0000	-	-	0s
H	0	0				9385.0000	8234.0000	12.3%	-	0s
	0	0	8567.0000	0	3	9385.0000	8567.0000	8.72%	-	0s
*	0	0			0	9210.0000	9210.0000	0.00%	-	0s

Explored 1 nodes (18 simplex iterations) in 0.01 seconds

Optimal solution found (tolerance 1.00e-04)

Best objective 9.210000e+03, best bound 9.210000e+03, gap 0.0000%

Key observations: “H” denotes a heuristic found the first feasible solution (incumbent = 9,385). The root LP relaxation gives a bound of 8,234. After cuts at node 0, the bound tightens to 8,567. The “*” marks the node where the optimal integer solution was found. The final line confirms a 0% gap — provably optimal.

5.3 Common Gurobi Options in AMPL

```
option solver gurobi;
option gurobi_options 'mipgap=0.01 timelim=300 outlev=1';
```

- **mipgap**: Stop when the relative gap drops below this threshold (e.g., 0.01 = 1%). Useful for large problems where proving exact optimality is too slow.
- **timelim**: Maximum solve time in seconds. The solver returns the best solution found within the limit.
- **outlev**: Output level (0 = silent, 1 = verbose log). Set to 1 when you want to inspect the solve process.
- **presolve**: Controls presolve intensity (0 = off, 1 = conservative, 2 = aggressive). Presolve simplifies the model before solving.

6. Computational Complexity in Practice

6.1 Scaling Behavior

While LP solve time scales roughly as $O(n^{2-3})$ with problem size (in practice), IP solve time can grow exponentially. A problem with 10 binary variables might solve in milliseconds; the same problem structure with 100 binary variables might take minutes; with 1,000 variables, it might be intractable without good cuts or structure.

6.2 What Makes an IP Easy or Hard?

Several factors affect practical difficulty:

- **LP relaxation tightness**: Tight relaxations (small integrality gap) allow more pruning.
- **Problem structure**: Network problems, set covering with good coefficients, and problems with “almost” TU matrices tend to be easier.
- **Constraint density**: Highly constrained problems may be easier (fewer feasible solutions) or harder (more complex LP relaxations), depending on the structure.
- **Objective coefficients**: Flat objectives (many solutions with similar values) make it hard to prune.
- **Symmetry**: Problems with many equivalent solutions (e.g., facility location with identical facilities) create large B&B trees unless symmetry-breaking constraints are added.

6.3 Demonstration Strategy

In the notebook, we will scale up the scheduling problem from 3 jobs to 5, 7, 9, ... jobs and observe how solve time and node count grow. We compare this to the LP relaxation, which scales gracefully. This makes the NP-hardness result tangible.

7. Practical Tips for Integer Programming

1. **Start with the LP relaxation.** Always solve it first. If the gap is large, consider reformulating.

2. **Use the tightest formulation you can.** Tighter Big-M values, tighter upper bounds in switching constraints, additional valid inequalities — all help.
 3. **Set appropriate tolerances.** For large problems, a 1% gap is often sufficient for practical decisions.
 4. **Use solver heuristics.** Providing a good initial solution (warm start) can dramatically speed up the search.
 5. **Exploit structure.** If your problem has network structure, use network-aware formulations. If it has symmetry, add symmetry-breaking constraints.
 6. **Monitor the solver log.** If the gap is not closing, consider reformulating, adding cuts, or increasing the time limit.
-

8. Summary

Concept	Key Idea
LP Relaxation	Drop integrality → get a bound on IP optimal
Integrality Gap	Measures quality of LP relaxation
Branch and Bound	Divide into subproblems, prune via LP bounds
Cutting Planes	Add inequalities that tighten LP relaxation
Branch and Cut	Modern hybrid: cuts + branching at every node
Solver output	Gap, nodes, cuts, time tell you the story

Next lecture: Duality and sensitivity analysis for LP (or as scheduled in your syllabus).